

Quantum Cryptography

...

Troy Frazier, Sam Gant, Aviral Verma, Ali Sbeiti
Team Fun(Key)

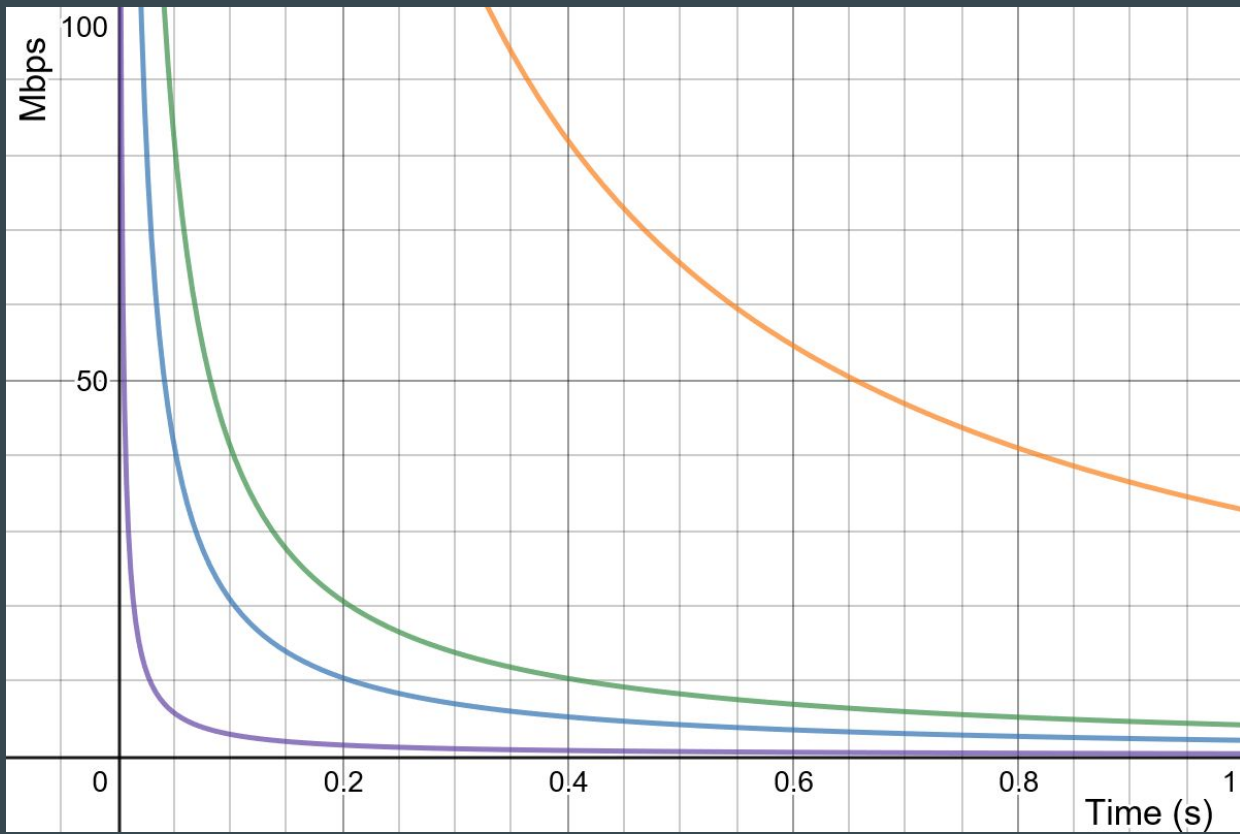
Previous Work

- Main focus was an abstract look at RSA with Quantum
 - Physical RSA
 - TCP with Quantum
 - VPNs
 - Quantum implementations for RSA
-
- Main focus this presentation is analysis

Issues with Quantum RSA

Physical Size of Key
Packet Fragmentation
Processing Time

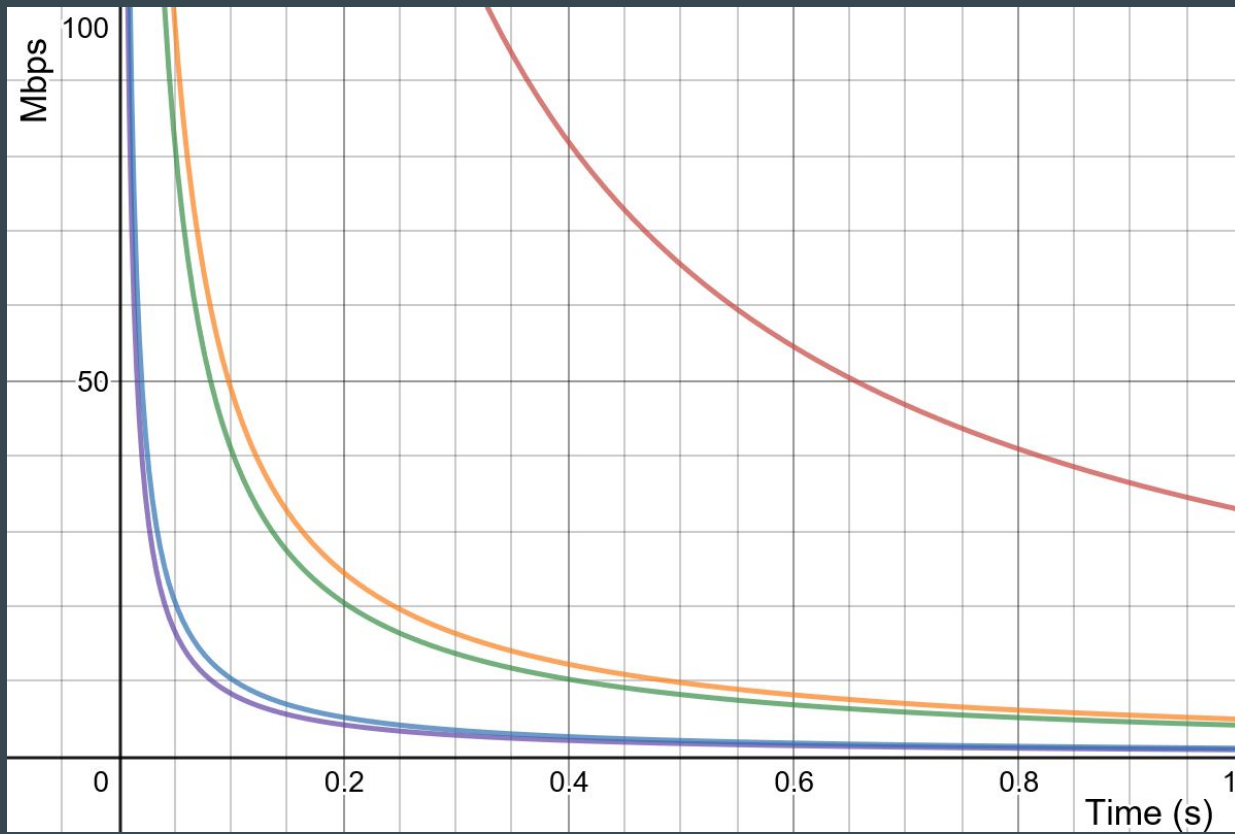
```
11424435476401506867567044883383791814261499396141697332137629902095226710
31123194426911850674917242872214865561662124663332057533500850377750025412
55105595000720511851353037621517745570690614060317083458676829125569878922
70827054725537341380266261086629566768253534191496715462772042174577040760
97998981829204733667344990344947304684926389055634422063423319901431077046
30053091522106113936247133944415792023708237087285788471771881132965683428
08245614040469346768582826209687281966641472944079507559701722657054821919
85864168195757560710141961915795825536920218100511861356285492426065902740
13110619290842200331217299165988774847016511584542910424526713922794482457
05442181699467714015649796844845339560244748396291199243632346426970786822
09316956856201809706493171304052301511003747970813123110656396679910666001
37584882747772589096723854053320782628031261934559416024706066944633346958
09912195102578575214875908992439300380994075945445270771255817382604391601
84243782302841753466779571476555456024747173876881477665773192923705560212
82569709497965687274449565001974078442788749994488443682110924676873252627
38513614297786164815734894508246677149837955873254148851980408514814698514
359503965612656156125
```



256 (2^8) vs. 2048 (2^{11}) vs. 4096 (2^{12})



4096 (2^{12}) vs. 32768 (2^{15}) vs. 131072 (2^{17})



→ New Hope: 4096 B

→ SIDH: 612 B

→ RSA 4096: 516 B

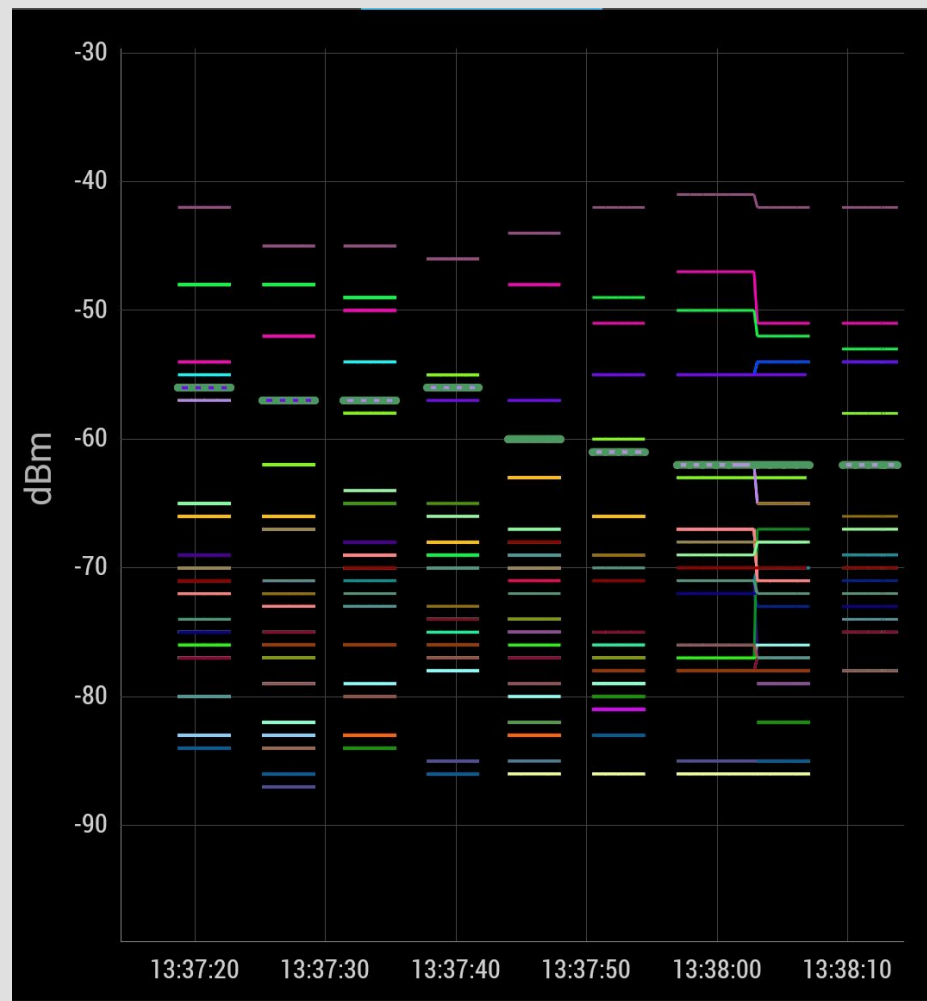
→ Elliptic Curve: 129 B

→ SPHINCS+: 104 B

Encryption Method Time Comparison

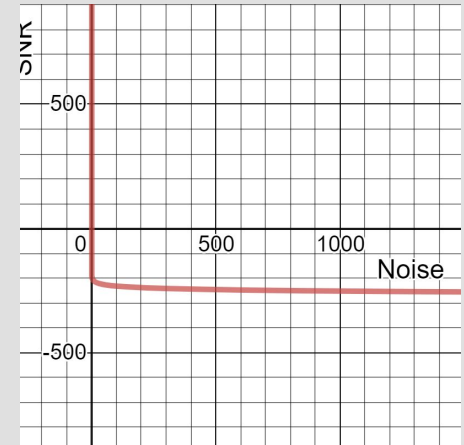
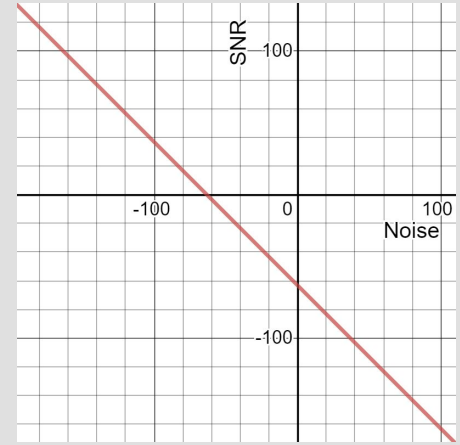
Bandwidth Measurements

- Active measurements of computers on a network. (Approx. 20 machines)
- Average bandwidth: -64.3 dBm
- Average was calculated by taking the average of each machine's dBm
- Download speed: 324 mbps



Derivations: SNR

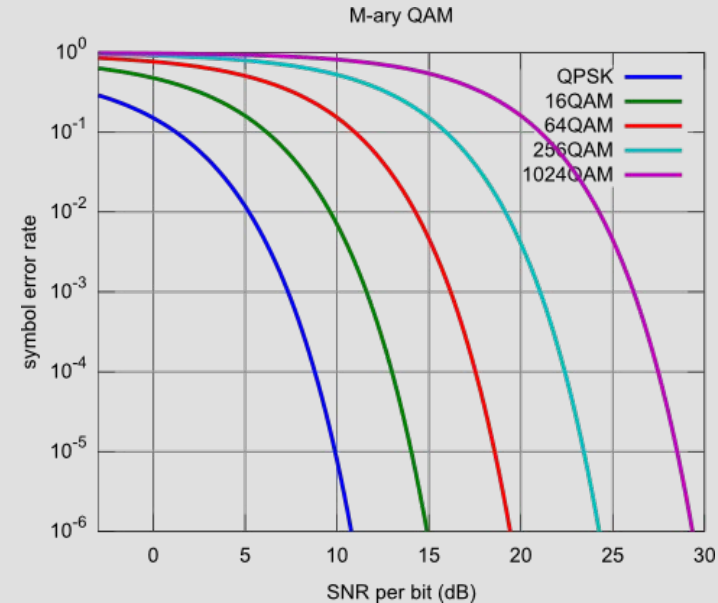
- SNR Calculated by $\text{SNR} = \text{Signal} - \text{noise}$
- Signal = -64.3 DbM, Noise = x DbM
- SNR > 30 Dbm is an optimal SNR
- In a Urban city setting, Noise floor is -92 Dbm (Not taken into account in graphs)
- Main focus is within wireless connections



Signal-To-Noise ratio of measured network in both forms of SNR

Bit Error Rate (BER)

- SNR at noise floor: 27.7 (Urban Environments)
 - One in every million bit is bad in worst case
 - Given 4096 bytes (32768 bits) we have the probability of approx. 3.2768% of a bit error
-
- The problem:
 - As keys get larger, bit error gets more likely
 - How do we fix error if encrypted (Avalanche Theory)



Measurements of BER from cloudrf

BENCHMARKING POST QUANTUM CRYPTO ALGORITHMS

Open Quantum Safe (OQS) project aims to support the development and prototyping of quantum-resistant cryptography.

OQS consists of two main lines of work: **liboqs**, an open source C library for quantum-resistant cryptographic algorithms, and prototype integrations into protocols and applications, including the widely used OpenSSL library.

Liboqs supports 7 PQ Key Encapsulation mechanisms (KEM) and 7 Signature Schemes. It provided us with an opportunity to look closely at the performance of these algorithms.

BENCHMARKING POST QUANTUM CRYPTO ALGORITHMS

“Lots and Lots” of algorithms as each algorithm has 5-6 different versions.

We focus our attention on few chosen versions that cover all variants of PQCrypto and are hardware independent.

Thus, the algorithms we analyse are New Hope CCA (512,1024); NTRUEncrypt hps4096821; Kyber-1024; Frodo KEM AES (976,1344); SIKE (p751, p751 Compressed).

Study the trade-offs between keysize and time.

BENCHMARKING POST QUANTUM CRYPTO ALGORITHMS

All the tests were run on i7-7700K Kabylake processor Laptop, with Frequency of 4.20 GHz.

NIST Security Level: NIST specifies levels 1-5 of strength for Quantum Algorithms.

Level 1 is equivalent to or exceeds the brute force strength of AES 128 algorithm.

Each subsequent level is an increment of 32 bits, thus; Level 2: AES 160, Level 3: AES 192; Level 4: AES 224 and Level 5: AES 256.

Algorithm	Operation	Iterations	Mean Time (<i>micro seconds</i>)	Mean CPU Cycles	NIST Security Level	Secret Key (<i>Bytes</i>)	Public Key (<i>Bytes</i>)
New Hope 512 CCA	keygen	361,506	27.662	110,128	1	928	1888
	encaps	237,091	42.178	167,348			
	decaps	201,556	49.614	198,522			
New Hope 1024 CCA	keygen	168,339	59.404	238,126	5	3680	1824
	encaps	109,175	91.596	368,592			
	decaps	92,796	107.763	429,890			
FrodoKEM 976	keygen	1,485	673.484	2,580,364	3	31296	15332
	encaps	1,218	821.294	3,286,240			
	decaps	1,318	759.002	3,089,774			
FrodoKEM 1344	keygen	859	1163.702	4,556,722	5	43088	21520
	encaps	680	1471.396	5,873,148			
	decaps	696	1436.108	5,602,548			

NTRU hps 4096821	keygen	577	1732.684	6,951,008	5	1592	1230
	encaps	9,752	102.548	410,196			
	decaps	3,870	258.396	1,049,032			
SIKE p751	keygen	8,939	1118.702	4,463,168	5*	644	564
	encaps	6,322	1581.568	6,291,482			
	decaps	5,120	1,953.07	7,795,180			
SIKE p751 Compressed	keygen	3,567	2803.751	11,203,758	5*	413	331
	encaps	2,899	3449.658	13,769,640			
	decaps	3,163	3162.254	12,627,716			
Kyber 1024	keygen	11,728	85.264	332,996	5	3168	1568
	encaps	10,075	99.256	397,004			
	decaps	5,888	169.838	673,442			

ANALYSIS

- New Hope was implemented by Google in Chrome browsers, it is well supported by its reasonable key size and Processing times (~100 micro s).
- NTRU and Kyber although have key sizes in range of 1000s of bytes, provide really good times. NTRU has larger keygen time but its encapsulation and decapsulation times are in 100s of micro seconds only.
- Kyber-1024 posts better mean times in all three operations but its secret key size is almost double that of NTRU and thrice that of New Hope.

ANALYSIS

- Frodo KEM is an Unstructured Lattice algorithm and has absurdly large key sizes, 15332 Bytes in Frodo 976 and 21520 Bytes in Frodo 1344.
- Their processing times for all three operations although much smaller than that of SIKE, do not compete with Structured Lattice Algorithms.
- The creators of Frodo KEM claim however that these times can be reduced by 40-60 times if implemented with AES accelerated hardware i.e., with AES NI Instructions enabled.

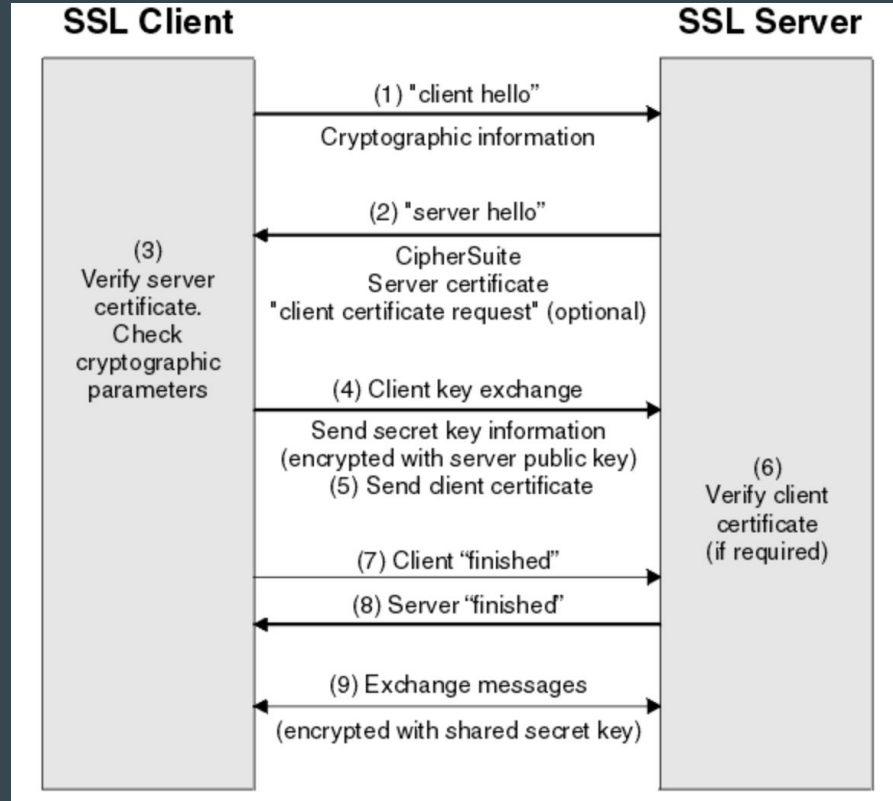
ANALYSIS

- SIKE is the solo entry in the Supersingular Isogeny (SI) group of algorithms. SIKE p751 has key sizes in the same range as RSA. (300-600 Bytes).
- The mean processing times for all three operations are significantly large. (1100-3500 micro s).
- SI algorithms are based on Elliptical Curve Cryptography (ECC), which has troubled cryptographers with its own fair share of problems in the years gone by.
- Thus, SL algorithms seem to be the best option for implementation in TLS 1.3 Protocol suite.

TLS Handshakes

- Clients and servers need to mutually generate a key that will be used to encrypt messages between the two endpoints.
- How do we securely transmit such a key without letting an adversary see it?
- For the longest time the solution was to distribute the key with asymmetric encryption
 1. Client application authenticates server by validating certificate with trusted Certificate Authority (CA).
 2. Client generates symmetric key, Usually AES-128/256.
 3. Client uses public key included in the server's certificate to encrypt the symmetric key, sends it to server.
 4. Client and server can now safely send messages to each other.

TLS Handshake Overview



Sourced From: IBM -
Overview of the SSL & TLS
handshake.

End of asymmetric encryption?

- Fear is quantum computing can more easily break asymmetric encryption methods like RSA and elliptic curve.
- Quantum bits can exist in superposition, theoretically allowing quantum computers to guess the keys much faster than any conventional computer.
- Could a solution be as easy as increasing the key size?
 - What impact on performance could this have?

Increasing the key sizes in RSA (1)

1024-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	72	128	54	1
Test 2	2	59	127	54	1
Test 3	4	61	126	52	0
Test 4	4	52	128	52	1
Test 5	4	84	129	51	1
Test 6	4	52	128	56	1
Test 7	2	56	131	55	1
Test 8	4	67	126	56	1
Test 9	4	52	127	53	0
Test 10	4	66	138	55	1
Average	3.6	62.1	128.8	53.8	0.8
2048-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	85	124	56	0
Test 2	5	59	123	50	0
Test 3	4	62	126	56	1
Test 4	4	62	127	56	0
Test 5	3	59	122	55	0
Test 6	3	57	129	49	1
Test 7	4	56	135	52	0
Test 8	4	60	131	52	0
Test 9	4	53	126	51	0
Test 10	4	62	128	54	0
Average	3.9	61.5	127.1	53.1	0.2

Increasing the key sizes in RSA (2)

4096-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	191	141	54	1
Test 2	4	54	138	52	0
Test 3	4	59	130	54	0
Test 4	4	63	143	52	0
Test 5	4	61	134	50	0
Test 6	4	54	132	52	0
Test 7	4	61	138	56	1
Test 8	2	53	157	50	0
Test 9	4	54	137	57	1
Test 10	4	55	128	50	1
Average	3.8	70.5	137.8	52.7	0.4

Increasing the key sizes in RSA (3)

8192-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	61	216	56	1
Test 2	2	55	183	65	1
Test 3	4	65	184	56	0
Test 4	2	67	181	56	0
Test 5	4	118	187	56	0
Test 6	4	54	207	51	1
Test 7	4	56	185	55	0
Test 8	2	74	199	52	0
Test 9	3	55	191	57	1
Test 10	4	56	186	53	0
Average	3.3	66.1	191.9	55.7	0.4
16384-bit key	DNS Lookup	TCP Connection	TLS Handshake	Server Processing	Content Transfer
Test 1	4	53	564	50	0
Test 2	4	60	621	58	0
Test 3	4	62	605	51	1
Test 4	2	52	502	54	0
Test 5	4	58	586	53	1
Test 6	4	53	551	54	1
Test 7	3	90	700	54	0
Test 8	4	68	529	51	1
Test 9	2	55	629	52	0
Test 10	2	53	603	53	1
Average	3.3	60.4	589	53	0.5

NewHope

Parameter Set	NEWHOPE512	NEWHOPE1024
Dimension n	512	1024
Modulus q	12289	12289
Noise parameter k	8	8
NTT parameter γ	10968	7
Decryption error probability	2^{-213}	2^{-216}
Claimed post-quantum bit-security	101	233
NIST Security Strength Category	1	5

Parameters: $q = 12289 < 2^{14}$, $n = 1024$

Error distribution: ψ_{16}

Alice (server)

$seed \xleftarrow{\$} \{0, 1\}^{256}$

$\mathbf{a} \leftarrow \text{Parse}(\text{SHAKE-128}(seed))$

$\mathbf{s}, \mathbf{e} \xleftarrow{\$} \psi_{16}^n$

$\mathbf{b} \leftarrow \mathbf{a}\mathbf{s} + \mathbf{e} \xrightarrow{(\mathbf{b}, seed)}$

$\mathbf{v}' \leftarrow \mathbf{u}\mathbf{s} \xleftarrow{(\mathbf{u}, \mathbf{r})}$

$\mathbf{v} \leftarrow \text{Rec}(\mathbf{v}', \mathbf{r})$

$\mu \leftarrow \text{SHA3-256}(\mathbf{v})$

Bob (client)

$\mathbf{s}', \mathbf{e}', \mathbf{e}'' \xleftarrow{\$} \psi_{16}^n$

$\mathbf{a} \leftarrow \text{Parse}(\text{SHAKE-128}(seed))$

$\mathbf{u} \leftarrow \mathbf{a}\mathbf{s}' + \mathbf{e}'$

$\mathbf{v} \leftarrow \mathbf{b}\mathbf{s}' + \mathbf{e}''$

$\mathbf{r} \xleftarrow{\$} \text{HelpRec}(\mathbf{v})$

$\mathbf{v} \leftarrow \text{Rec}(\mathbf{v}, \mathbf{r})$

$\mu \leftarrow \text{SHA3-256}(\mathbf{v})$

Implementation of NewHope

- Google implemented CECpq2 on all experimental branches of Chrome.
- CECpq2 is a Google developed key exchange protocol that combines X25519 with NewHope quantum resistant RLWE.
- About 15 ms increase in handshake time.

X25519	TLS Time(ms)
Run 1	59
Run 2	55
Run 3	54
Run 4	55
Run 5	54
Average	55.4

CECPQ2	TLS Time(ms)
Run 1	72
Run 2	71
Run 3	68
Run 4	74
Run 5	68
Average	70.6

References

- https://cloudrf.com/bit_error_rate
- <https://sciencing.com/how-to-calculate-signal-to-noise-ratio-13710251.html>
- <https://eyesaa.com/wi-fi-signal-strength/>
- https://www.ibm.com/support/knowledgecenter/en/SSFKSI_7.1.0/com.ibm.mq.doc/sy10660_.htm
- https://newhopecrypto.org/data/NewHope_2019_07_10.pdf
- <https://github.com/open-quantum-safe/liboqs>
- https://en.wikipedia.org/wiki/Post-quantum_cryptography#
- <http://sphincs.org/>
- <https://www.newhopecrypto.org/>
- https://en.wikipedia.org/wiki/Supersingular_isogeny_key_exchange
- <https://www.imperialviolet.org/2018/04/11/pqconftls.html>